HDOC Day-47 Activity

Ceil Index in a Sorted Variable-Length Array Using Pointers

Name	Gurveer Singh Cheema
SAF ID	590032925
Batch	15

Aim

To read a sorted variable-length integer array and a target value, pass the array to **ceil_index()** using a pointer, and find the index of the smallest array element that is greater than or equal to the target.

Requirements

1. Read size and elements of a sorted VLA **arr[]** in ascending order.
2. Read an integer target.
3. Pass the array by address to **ceil_index()**.
4. Find the smallest element satisfying **element >= target**.
5. If that value occurs more than once, return its first occurrence.
6. Display the ceil element and its index.
7. Display **-1** if no array element is greater than or equal to target.

Algorithm

1. Read the size of the array.
2. Create the variable-length array arr[size].
3. Read the already sorted elements.
4. Read target.
5. Pass arr, size, and target to ceil_index().
6. Scan from index 0 because the array is in ascending order.
7. The first element for which arr[i] >= target is automatically the smallest suitable element and its first occurrence.
8. Return that index immediately.
9. If no such element exists, return -1.

C Program

```
#include <stdio.h>

int ceil_index(int *arr, int size, int target)
{
    int i;

    for (i = 0; i < size; i++)
    {
        if (*(arr + i) >= target)
            return i;
    }

    return -1;
}

int main()
{
    int size, i, target, index;
```

```
    printf("Enter size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d sorted elements in ascending order:\n", size);

    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);

    printf("Enter target: ");
    scanf("%d", &target);

    index = ceil_index(arr, size, target);

    if (index == -1)
    {
        printf("-1\n");
    }
    else
    {
        printf("Ceil of %d: %d at index %d\n",
               target, arr[index], index);
    }

    return 0;
}
```

Sample Outputs

```
Example 1
Enter size of array: 6
Enter 6 sorted elements in ascending order:
5 7 7 8 8 10
Enter target: 6
Ceil of 6: 7 at index 1

Example 2
Enter size of array: 6
Enter 6 sorted elements in ascending order:
5 7 7 8 9 10
Enter target: 8
Ceil of 8: 8 at index 3

Example 3
Enter size of array: 6
Enter 6 sorted elements in ascending order:
5 7 7 8 8 10
Enter target: 11
-1
```

Function Explanation

ceil_index(int *arr, int size, int target): The function receives the base address of the array through the pointer arr. It scans from left to right and returns the first index whose value is greater than or equal to target. Because the array is sorted, this is the smallest qualifying value. Scanning from the beginning also guarantees the first occurrence when duplicates exist.

Pointer Concept Used

arr[i] and ***(arr + i)** refer to the same element. Passing arr to ceil_index() passes the address of the first element, so the function can access the original array without making a copy.

Easy Trick to Remember

Ceil = first value >= target.
Start from the left -> find the first value that is equal to or bigger than target -> return its index.
Example: 5 7 7 8 8 10, target = 6 -> first value >= 6 is 7 -> index 1.

Important Cases

Target exactly exists: return its first occurrence.
Target lies between two values: return the next larger value.
Target is smaller than all values: index 0 is the ceil.
Target is larger than every value: return -1.

Complexity

This simple pointer-based linear search takes **O(n)** time and **O(1)** extra space. Since the input is sorted, a binary-search version could reduce the search time to O(log n), but the linear version is straightforward and satisfies the activity.

Result

Thus, the sorted variable-length array was passed using a pointer and the ceil element and its first index were found successfully. The program correctly returns -1 when no ceil element exists.

Quick Viva Questions

1. **What is the ceil of a target?** The smallest array element greater than or equal to the target.
2. **Why does the first match work?** The array is sorted in ascending order.
3. **What happens with duplicate ceil values?** The first occurrence is returned.
4. **What does *(arr + i) mean?** The array element at index i.
5. **When does the function return -1?** When no element is greater than or equal to target.
6. **For 5 7 7 8 8 10 and target 6, what is the answer?** Ceil is 7 at index 1.